

UOS - Capstone

Week 2 -Variational AutoEncoders

박상훈 ¹

2026학년도 2학기

¹Gwanak Lab Co., Ltd., Korea
E. sanghun.park@gwanaklab.co.kr

Notations

Latent Representation & AutoEncoder

Variational AutoEncoders

β -VAE

Latent Space Visualization

Tabular VAE and Implementation

Notations

데이터 공간

- $\mathbf{x} \in \mathbb{R}^p$: 관측 데이터 포인트 (p 개의 특성(feature)을 가진 벡터)
- $\{\mathbf{x}^{(i)}\}_{i=1}^n$: n 개의 학습 데이터
- $p^*(\mathbf{x})$: 우리가 모사하고자 하는 실제 데이터 분포

잠재 공간 (Latent Space)

- $\mathbf{z} \in \mathbb{R}^d$, $d < p$: 데이터를 압축한 저차원 잠재 변수(latent variable)
- $p(\mathbf{z})$: 잠재 공간의 사전 분포(prior distribution) ↪ 주로 표준 정규분포 $\mathcal{N}(\mathbf{0}, \mathbf{I})$ 사용

신경망 파라미터

- ϕ : 인코더(Encoder) 파라미터 — 데이터 $\mathbf{x}$ 를 잠재 변수 $\mathbf{z}$ 로 변환
- θ : 디코더(Decoder) 파라미터 — 잠재 변수 $\mathbf{z}$ 로부터 $\mathbf{x}$ 를 복원

확률 모델

- $q(\mathbf{z} | \mathbf{x}; \phi)$: 인코더가 정의하는 **사후 분포(posterior or encoder)**
⇒ “데이터 $\mathbf{x}$ 가 주어졌을 때, 잠재 변수 $\mathbf{z}$ 가 따르는 분포”
- $p(\mathbf{x} | \mathbf{z}; \theta)$: 디코더가 정의하는 **조건부 분포(likelihood)**
⇒ “잠재 변수 $\mathbf{z}$ 가 주어졌을 때, 데이터 $\mathbf{x}$ 를 재구성하는 분포”

자주 사용하는 연산자

- $\mathbb{E}_q[\cdot]$: 분포 q 에 대한 **기댓값(expectation)**
- $\mathcal{KL}(q \| p)$: 두 분포 q 와 p 사이의 **KL 발산(KL divergence)**

$$\mathcal{KL}(q \| p) := \mathbb{E}_q \left[\log \frac{q(\mathbf{z})}{p(\mathbf{z})} \right] = \int q(\mathbf{z}) \log \frac{q(\mathbf{z})}{p(\mathbf{z})} d\mathbf{z} \geq 0$$

⇒ 두 분포의 차이를 측정; $q = p$ 이면 $\mathcal{KL} = 0$

- $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$: 평균 $\boldsymbol{\mu}$, 공분산 $\boldsymbol{\Sigma}$ 인 **정규분포**

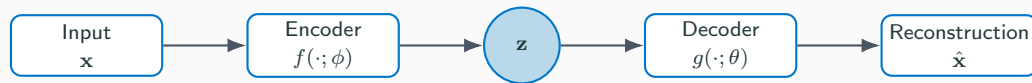
Latent Representation & AutoEncoder

- 어떠한 관측 데이터(observed data) $\mathbf{x} \in \mathbb{R}^p$ 는 고차원이며, 중복되거나 직접 해석하기 어려운 정보를 포함합니다.
- 데이터의 주요 변화를 설명하는 더 낮은 차원의 변수(latent variable) $\mathbf{z} \in \mathbb{R}^d$ 를 가정합니다.

Latent representation $\mathbf{z}$

$$\mathbf{x} \rightarrow \mathbf{z}, \quad d < p$$

- 좋은 $\mathbf{z}$ 는 원본 데이터의 모든 값을 복사하는 것이 아니라, 원본을 설명하고 복원하는 데 필요한 정보를 담습니다.
- 정답으로 주어진 latent variable은 없으므로, 원본을 다시 복원하는 문제를 통해 $\mathbf{z}$ 를 학습합니다.



$$\begin{aligned}\mathbf{z} &= f(\mathbf{x}; \phi), \\ \hat{\mathbf{x}} &= g(\mathbf{z}; \theta) = g(f(\mathbf{x}; \phi); \theta).\end{aligned}$$

- Encoder는 입력을 잠재표현으로 변환하고, Decoder는 잠재표현으로부터 입력을 복원합니다.
- $d < p$ 인 bottleneck을 통과하면서 복원에 필요한 정보가 $\mathbf{z}$ 에 압축됩니다.

- Encoder와 Decoder는 reconstruction loss가 작아지도록 함께 학습됩니다.

$$\min_{\phi, \theta} \mathcal{L}_{\text{AE}}(\phi, \theta) = \frac{1}{n} \sum_{i=1}^n \ell(\mathbf{x}^{(i)}, g(f(\mathbf{x}^{(i)}; \phi); \theta)). \quad (1)$$

연속형 입력 (MSE)

$$\ell(\mathbf{x}, \hat{\mathbf{x}}) = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2$$

범주형 입력 (Cross-Entropy)

$$\ell(\mathbf{x}, \hat{\mathbf{x}}) = - \sum_j [\mathbf{x}_j \log \hat{\mathbf{x}}_j + (1 - \mathbf{x}_j) \log(1 - \hat{\mathbf{x}}_j)]$$

Representation learning: $\hat{\mathbf{x}}$ 를 잘 만들기 위해 필요한 정보를 $\mathbf{z}$ 에 남기는 과정

- AutoEncoder는 각 입력을 하나의 **결정론적(deterministic)** 점 $\mathbf{z} = f(\mathbf{x}; \phi)$ 으로 변환합니다.
- 잠재 공간에서 각 $\mathbf{z}$ 가 어디에 위치해야 하는지에 대한 **확률적 구조가 없습니다.**
- 학습 데이터에 없는 임의의 $\mathbf{z}$ 를 샘플링하면, Decoder가 의미 없는 결과를 만들 수 있습니다.

핵심 질문

학습 완료 후 새로운 데이터를 **생성**하려면 어떻게 해야 할까?
즉, 어떤 $\mathbf{z}$ 를 샘플링해야 의미 있는 데이터가 만들어지는가?

Variational AutoEncoders

Purpose of VAEs

- VAE는 입력을 하나의 점이 아니라 **잠재 확률분포**로 변환합니다.

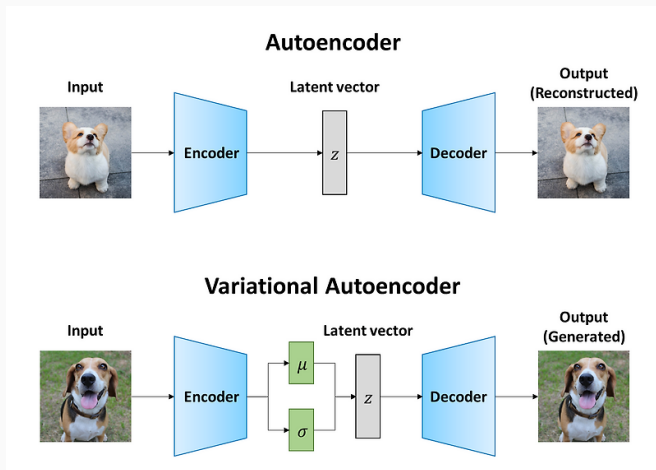


Figure 1: Overview of AutoEncoders and Variational AutoEncoders

인코더 분포 — 점이 아닌 분포를 출력

$$q(\mathbf{z} \mid \mathbf{x}; \phi) := \mathcal{N}(\mathbf{z} \mid \boldsymbol{\mu}(\mathbf{x}; \phi), \text{diag}(\boldsymbol{\sigma}^2(\mathbf{x}; \phi))) .$$

- 인코더는 하나의 점이 아닌 **평균** $\boldsymbol{\mu}(\mathbf{x}; \phi)$ 와 **분산** $\boldsymbol{\sigma}^2(\mathbf{x}; \phi)$ 벡터를 출력합니다.

Reparameterization trick (역전파를 가능하게 하는 핵심 기법)

$$\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad \mathbf{z} = \boldsymbol{\mu}(\mathbf{x}; \phi) + \boldsymbol{\sigma}(\mathbf{x}; \phi) \odot \boldsymbol{\epsilon}, \quad (2)$$

where $\boldsymbol{\sigma}(\mathbf{x}; \phi) := \sqrt{\boldsymbol{\sigma}^2(\mathbf{x}; \phi)}$ (element-wise), $\odot$: 원소별 곱(Hadamard product).

- 무작위성을 $\boldsymbol{\epsilon}$ 으로 분리하면 $\mathbf{z}$ 가 $\boldsymbol{\mu}(\cdot; \phi)$ 와 $\boldsymbol{\sigma}(\cdot; \phi)$ 의 **미분 가능한 함수**가 됩니다.

디코더 분포 — 잠재 변수로부터 데이터를 복원

$$p(\mathbf{x} \mid \mathbf{z}; \theta) := \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}(\mathbf{z}; \theta), \sigma^2 \cdot \mathbf{I}) .$$

- $\boldsymbol{\mu}(\cdot; \theta) : \mathbb{R}^d \rightarrow \mathbb{R}^p$ 는 Decoder 신경망으로, 잠재 변수 $\mathbf{z}$ 로부터 복원된 데이터의 **평균**을 출력합니다.
- σ^2 는 스칼라 분산 (단순화를 위해 학습하지 않거나 고정)합니다.
- 실제 구현에서는 MSE loss가 $-\log p(\mathbf{x} \mid \mathbf{z}; \theta)$ 에 해당합니다.

Maximum Likelihood and Its Difficulty

- 생성모형의 목표는 관측 데이터의 **marginal log-likelihood**를 최대화하는 것입니다.

$$\begin{aligned}\max_{\theta} \log p(\mathbf{x}; \theta) &= \log \int p(\mathbf{x}, \mathbf{z}; \theta) d\mathbf{z} \\ &= \log \int \underbrace{p(\mathbf{x} | \mathbf{z}; \theta)}_{\text{Decoder}} \underbrace{p(\mathbf{z})}_{\text{Prior}} d\mathbf{z}.\end{aligned}\tag{3}$$

- 문제:** 신경망 Decoder를 사용하면 위 **적분이 계산 불가능(intractable)**합니다.

$$p(\mathbf{z} | \mathbf{x}; \theta) = \frac{p(\mathbf{x} | \mathbf{z}; \theta) p(\mathbf{z})}{\underbrace{\int p(\mathbf{x} | \mathbf{z}; \theta) p(\mathbf{z}) d\mathbf{z}}_{\text{계산 불가능}}}$$

해결책: 계산 가능한 분포 $q(\mathbf{z}|\mathbf{x}; \phi)$ 로 사후 분포를 근사하고,
다루기 쉬운 **하한(Evidence Lower BOund, ELBO)**을 최적화합니다.

- 임의의 분포 $q(\mathbf{z} \mid \mathbf{x}; \phi)$ 를 도입하여 log-likelihood를 다시 씁니다.

$$\begin{aligned}\log p(\mathbf{x}; \theta) &= \log \int q(\mathbf{z} \mid \mathbf{x}; \phi) \cdot \frac{p(\mathbf{x}, \mathbf{z}; \theta)}{q(\mathbf{z} \mid \mathbf{x}; \phi)} d\mathbf{z} \\ &= \log \mathbb{E}_{q(\mathbf{z} \mid \mathbf{x}; \phi)} \left[\frac{p(\mathbf{x}, \mathbf{z}; \theta)}{q(\mathbf{z} \mid \mathbf{x}; \phi)} \right].\end{aligned}\tag{4}$$

- Jensen's inequality를 적용합니다.

$$\log p(\mathbf{x}; \theta) \geq \mathbb{E}_{q(\mathbf{z} \mid \mathbf{x}; \phi)} [\log p(\mathbf{x}, \mathbf{z}; \theta) - \log q(\mathbf{z} \mid \mathbf{x}; \phi)].\tag{5}$$

- $p(\mathbf{x}, \mathbf{z}; \theta) = p(\mathbf{x} | \mathbf{z}; \theta) p(\mathbf{z})$ 를 대입하여 정리합니다.

$$\begin{aligned} \mathcal{L}_{\text{ELBO}}(\mathbf{x}; \theta, \phi) &:= \underbrace{\mathbb{E}_{q(\mathbf{z}|\mathbf{x};\phi)}[\log p(\mathbf{x} | \mathbf{z}; \theta)]}_{\text{복원 오차항 (Reconstruction)}} \\ &\quad - \underbrace{\mathcal{KL}(q(\mathbf{z} | \mathbf{x}; \phi) \| p(\mathbf{z}))}_{\text{정규화 항 (Regularization)}} \leq \log p(\mathbf{x}; \theta). \end{aligned} \quad (6)$$

ELBO = Reconstruction – Latent Regularization
 ELBO 최대화 $\Leftrightarrow$ likelihood 증가 + $q(\mathbf{z}|\mathbf{x}; \phi) \approx p(\mathbf{z}|\mathbf{x}; \theta)$

Source: D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," ICLR, 2014.

Why Is ELBO a Lower Bound?

- Log-likelihood는 ELBO와 사후 근사 오차로 **정확히** 분해됩니다.

$$\log p(\mathbf{x}; \theta) = \mathcal{L}_{\text{ELBO}}(\mathbf{x}; \theta, \phi) + \underbrace{\mathcal{KL}(q(\mathbf{z} | \mathbf{x}; \phi) \| p(\mathbf{z} | \mathbf{x}; \theta))}_{\geq 0}. \quad (7)$$

- KL divergence는 항상 ≥ 0 이므로 $\mathcal{L}_{\text{ELBO}} \leq \log p(\mathbf{x}; \theta)$.
- ELBO를 최대화하면 두 가지가 동시에 달성됩니다.
 - (1) $\log p(\mathbf{x}; \theta)$ (likelihood) 증가
 - (2) $q(\mathbf{z} | \mathbf{x}; \phi)$ 가 실제 사후 분포 $p(\mathbf{z} | \mathbf{x}; \theta)$ 에 가까워짐

- 실제 구현에서는 ELBO의 음수를 loss로 최소화합니다.

$$\min_{\theta, \phi} \mathcal{L}_{\text{VAE}} = \underbrace{-\mathbb{E}_{q(\mathbf{z}|\mathbf{x};\phi)}[\log p(\mathbf{x} | \mathbf{z}; \theta)]}_{\mathcal{L}_{\text{recon}}} + \underbrace{\mathcal{KL}(q(\mathbf{z} | \mathbf{x}; \phi) \| p(\mathbf{z}))}_{\mathcal{L}_{\text{KL}}}. \quad (8)$$

Gaussian prior일 때 KL 항의 닫힌 식

$$\mathcal{L}_{\text{KL}} = \mathcal{KL}(\mathcal{N}(\boldsymbol{\mu}(\mathbf{x}; \phi), \text{diag}(\boldsymbol{\sigma}^2(\mathbf{x}; \phi))) \| \mathcal{N}(\mathbf{0}, \mathbf{I})) = -\frac{1}{2} \sum_{k=1}^d (1 + \log \sigma_k^2(\mathbf{x}; \phi) - \mu_k^2(\mathbf{x}; \phi) - \sigma_k^2(\mathbf{x}; \phi)) \quad (9)$$

- 인코더를 Gaussian으로 가정하면 KL 항을 **적분 없이** 계산할 수 있습니다.

수식

$$\begin{aligned}(\boldsymbol{\mu}, \log \boldsymbol{\sigma}^2) &= \text{Encoder}(\mathbf{x}; \phi), \\ \mathbf{z} &= \boldsymbol{\mu} + \exp\left(\frac{1}{2} \log \boldsymbol{\sigma}^2\right) \boldsymbol{\epsilon}, \\ \hat{\mathbf{x}} &= \text{Decoder}(\mathbf{z}; \theta), \\ \mathcal{L} &= \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{KL}}.\end{aligned}$$

```
mu, logvar = encoder(x)

std = torch.exp(0.5 * logvar)
eps = torch.randn_like(std)
z = mu + std * eps

x_hat = decoder(z)
loss = recon_loss(x_hat, x) \
      + kl_loss(mu, logvar)

loss.backward()
optimizer.step()
```

생성(Generation): $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 를 샘플링한 뒤 Decoder만 사용합니다.

β -VAE

Balance between Reconstruction and Regularization

- VAE 목적함수의 두 항은 서로 **상충(trade-off)** 관계에 있습니다.

$$\mathcal{L}_{\text{VAE}} = \underbrace{\mathcal{L}_{\text{recon}}}_{\text{복원 품질}} + \underbrace{\mathcal{L}_{\text{KL}}}_{\text{잠재 공간 정규화}} \quad (10)$$

$\mathcal{L}_{\text{KL}}$ 이 너무 강하면

- 잠재 분포가 $\mathcal{N}(\mathbf{0}, \mathbf{I})$ 에 강하게 끌림
- 데이터 정보가 $\mathbf{z}$ 에 충분히 담기지 않음
- 복원 품질 저하 (**posterior collapse**)

$\mathcal{L}_{\text{KL}}$ 이 너무 약하면

- 복원은 잘 됨
- 잠재 공간에 구조가 없어짐
- 임의 샘플링 시 의미 없는 결과 생성

핵심 질문

$\mathcal{L}_{\text{KL}}$ 의 강도를 태스크에 맞게 **유연하게 조절**할 수 있을까?

- β -VAE는 KL 항에 가중치 $\beta > 0$ 를 곱하여 정규화 강도를 조절합니다.

$$\min_{\theta, \phi} \mathcal{L}_{\beta\text{-VAE}} = \underbrace{-\mathbb{E}_{q(\mathbf{z}|\mathbf{x};\phi)}[\log p(\mathbf{x} | \mathbf{z}; \theta)]}_{\mathcal{L}_{\text{recon}}} + \beta \cdot \underbrace{\mathcal{KL}(q(\mathbf{z} | \mathbf{x}; \phi) \| p(\mathbf{z}))}_{\mathcal{L}_{\text{KL}}} \quad (11)$$

β 값	의미
$\beta = 1$	기본 VAE와 동일
$\beta > 1$	KL 강화 $\rightarrow$ 잠재 공간이 $\mathcal{N}(\mathbf{0}, \mathbf{I})$ 에 가까워짐
$0 < \beta < 1$	복원 품질 우선, 정규화 완화

- $\beta > 1$ 이면 각 잠재 차원이 **독립적인 의미**를 갖도록 유도됩니다 (**Disentanglement**).

Source: I. Higgins et al., " β -VAE: Learning Basic Visual Concepts with a Constrained Variational Framework," ICLR, 2017.

- β -VAE는 다음 제약 최적화 문제의 Lagrangian 완화(relaxation)로 해석할 수 있습니다.

$$\begin{aligned} \max_{\phi, \theta} \quad & \mathbb{E}_{q(\mathbf{z}|\mathbf{x};\phi)}[\log p(\mathbf{x} | \mathbf{z}; \theta)] \\ \text{s.t.} \quad & \mathcal{KL}(q(\mathbf{z} | \mathbf{x}; \phi) \parallel p(\mathbf{z})) \leq \epsilon \end{aligned}$$

- 즉, 복원 품질을 최대화하되, 잠재 분포가 사전 분포에서 너무 벗어나지 않도록 제약합니다.
- β 는 이 제약의 **Lagrange multiplier**에 해당하며, ϵ 이 작을수록 β 가 커집니다.
- β 가 클수록 잠재 공간이 더 압축되고, 각 차원이 독립적이고 해석 가능한 요인을 포착합니다.

β 는 데이터와 태스크에 따라 조정해야 하는 중요한 하이퍼파라미터입니다.

Latent Space Visualization

- 학습 후 인코더가 데이터를 어떻게 압축하는지 **직관적으로 확인할 수** 있습니다.
- 잠재 변수 $\mathbf{z} \in \mathbb{R}^d$ 는 일반적으로 $d > 2$ 이므로 직접 산점도를 그릴 수 없습니다.
- **주성분 분석(PCA)**을 이용하여 2차원으로 투영합니다.

시각화 절차

$$\{\mathbf{x}^{(i)}\} \xrightarrow{\text{Encoder}} \{\mathbf{z}^{(i)}\} \xrightarrow{\text{PCA}} \{\tilde{\mathbf{z}}_{2D}^{(i)}\} \xrightarrow{\text{산점도}} \text{시각화}$$

- 레이블(class)별로 색깔을 달리하면, 잠재 공간에서 클래스가 얼마나 잘 분리되는지 확인할 수 있습니다.

- **AE**: 잠재 공간에 **확률적 제약이 없으므로**, 인코더가 데이터에 맞게 자유롭게 배치합니다.
⇒ 클러스터 사이에 **빈 공간(hole)**이 생기고, 임의의 $\mathbf{z}$ 를 샘플링하면 의미 없는 결과가 나옵니다.
- **VAE** ($\beta = 1$): 잠재 분포가 $\mathcal{N}(\mathbf{0}, \mathbf{I})$ 에 가깝도록 정규화합니다.
⇒ 잠재 공간이 **연속적이고 매끄럽게** 분포하여, 임의의 $\mathbf{z} \sim p(\mathbf{z})$ 에서도 의미 있는 샘플 생성.
- **β -VAE**: 더 강한/약한 정규화.
⇒ 잠재 공간에 어떤 변화가 생기는지 확인합니다.

AE

클러스터 사이에 빈 공간,
불연속적

VAE ($\beta = 1$)

연속적이고 매끄러운
잠재 공간

β -VAE

더 균일하고
구조화된 잠재 공간

Tabular VAE and Implementation

VAE의 학습 원리는 그대로이며, 입력 $\mathbf{x}$ 를 구성하는 방식이 달라집니다.

Image

$$\mathbf{x} \in [0, 1]^{H \times W \times C}$$

- 모든 픽셀이 동일한 자료형(실수)
- 배열을 tensor로 바로 입력 가능

Tabular data

$$\mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_p)$$

- 수치형(continuous) · 범주형(categorical) 열이 혼합
- 각 열을 변환하여 하나의 숫자 벡터로 구성

Tabular preprocessing: categorical encoding + continuous transformation

- 정형데이터의 변수는 유형에 따라 다른 전처리 방식을 적용합니다.
- I_C : 연속형(continuous) 변수의 인덱스 집합, I_D : 범주형(discrete) 변수의 인덱스 집합

변수 유형	변환 방법	수치 표현
순서형(Ordinal)	Ordinal encoding	$c_1 < c_2 < \dots < c_K \mapsto 0, 1, \dots, K - 1$
명목형(Nominal)	One-hot encoding	$c_k \mapsto (0, \dots, 1, \dots, 0)$
연속형(Continuous)	Quantile / Normal transform	$u = \hat{F}_j(\mathbf{x}_j), \quad \mathbf{x}_j = \Phi^{-1}(u)$

모든 열을 신경망이 입력받을 수 있는 숫자 벡터로 변환하고,
생성 후에는 원래 자료형을 복원합니다.

- 변환된 정형데이터에 동일한 VAE 구조를 적용합니다.
- 디코더 분포는 변수 유형별로 분리하여 정의합니다.
- T_j : $\mathbf{x}_j$ 범주형 변수 인덱스, $j \in I_D$ 의 범주 수

$$q(\mathbf{z} \mid \mathbf{x}; \phi) \quad := \quad \mathcal{N}(\mathbf{z} \mid \boldsymbol{\mu}(\mathbf{x}; \phi), \text{diag}(\boldsymbol{\sigma}^2(\mathbf{x}; \phi))) ,$$
$$p(\mathbf{x} \mid \mathbf{z}; \theta) \quad := \quad \prod_{j \in I_C} \mathcal{N}(\mathbf{x}_j \mid D_j(\mathbf{z}; \theta), s_j^2(\mathbf{z}; \theta)) \cdot \prod_{j \in I_D} \prod_{l=1}^{T_j} \pi_{jl}(\mathbf{z}; \theta)^{\mathbb{I}(\mathbf{x}_j=l)} \quad (12)$$

(13)

Notation. $\text{diag}(a)$: $a \in \mathbb{R}^d$ 의 원소를 대각 성분으로 갖는 대각행렬. $\boldsymbol{\mu}(\cdot; \phi) : \mathbb{R}^p \rightarrow \mathbb{R}^d$ 와 $\boldsymbol{\sigma}^2(\cdot; \phi) : \mathbb{R}^p \rightarrow \mathbb{R}_{++}^d$ 는 Gaussian 인코더의 평균·분산 벡터. $D_j(\cdot; \theta) : \mathbb{R}^d \rightarrow \mathbb{R}$ 와 $s_j^2(\cdot; \theta) : \mathbb{R}^d \rightarrow \mathbb{R}_{++}$ 는 연속형 변수 j 의 디코더 평균·분산. $\pi_j(\cdot; \theta) : \mathbb{R}^d \rightarrow [0, 1]^{T_j}$ 는 범주형 변수 j 의 다항 분포 파라미터이며, π_{jl} 은 그 l 번째 원소 ($\sum_{l=1}^{T_j} \pi_{jl}(\mathbf{z}; \theta) = 1$). $\mathbb{I}(\cdot)$: 지시함수.

- 앞서 정의한 β 를 적용한 최종 목적함수:

$$\mathcal{L}_{\text{TVAE}} = \underbrace{-\mathbb{E}_{q(\mathbf{z}|\mathbf{x};\phi)}[\log p(\mathbf{x} | \mathbf{z}; \theta)]}_{\mathcal{L}_{\text{recon}}} + \beta \mathcal{KL}(q(\mathbf{z} | \mathbf{x}; \phi) \| p(\mathbf{z})) \quad (14)$$

$$= \underbrace{\sum_{j \in I_C} \left[\frac{1}{2} \log(2\pi s_j^2(\mathbf{z}; \theta)) + \frac{(\mathbf{x}_j - D_j(\mathbf{z}; \theta))^2}{2s_j^2(\mathbf{z}; \theta)} \right] + \sum_{j \in I_D} \sum_{l=1}^{T_j} [-\mathbb{I}(\mathbf{x}_j = l) \log \pi_{jl}(\mathbf{z}; \theta)]}_{\mathcal{L}_{\text{recon}}} - \underbrace{\frac{\beta}{2} \sum_{k=1}^d (1 + \log \sigma_k^2(\mathbf{x}; \phi) - \mu_k^2(\mathbf{x}; \phi) - \sigma_k^2(\mathbf{x}; \phi))}_{\beta \cdot \mathcal{L}_{\text{KL}}} . \quad (15)$$

Source: L. Xu et al., "Modeling Tabular Data using Conditional GAN," NeurIPS, 2019.

1. β -VAE에서 β 를 고정하지 않고 학습과정에서 유동적으로 바뀌는 학습 방식을 많이 사용한다. 예상 및 기대되는 효과에 대해서 생각해보자.
2. VAE의 Gaussian 가정에서 어떤 부분이 한계라고 생각되는가?
3. 일반적으로 VAE는 Blur한 이미지 샘플을 생성하는 것으로 알려진다. 그 이유는 무엇인가?